

PRINTED ELECTRONICS

Imagine being able to make electrically-functional components by 'printing' them on materials such as plastic, silicon and ceramics, using normal printing techniques like screen printing, offset lithography and inkjet! Electronics manufactured using such techniques would be very cheap, and especially suitable for low-performance items such as smart tags and flexible displays.

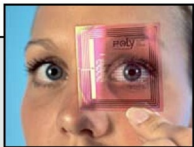
Printed electronics are typically produced by rapidly printing multiple conductive, insulating and semi-conductive layers to build active or passive components such as thin-film transistors or resistors, which in turn form electronic circuits.

As mentioned earlier, regular printing processes in slightly modified forms can be used for this, the special aspect being the inks used for printing. The inks are usually electrically-functional materials composed of carbon-based compounds. Organic as well as inorganic materials can be used, the key criteria being that the material should be available in a liquid form. Several such organic materials are available to be used as conductors, semi-conductors or insulators. Inorganic materials used are normally dispersions of metallic micro- and nano-particles. A huge break for printed electronics came with the discovery of conjugated polymers, which even won the Nobel Prize in chemistry in the year 2000, and their development into soluble

materials.

There are companies (such as Kovio Inc and Merck) that have been making printed semiconductor devices since 2001, and of late they seem to be venturing into mass production as well. Printing equipment are available from companies such as IBM and Xerox, while the substrates and inks can be had from 3M, Tetra Pak, Mitsubishi Chemicals, etc.

As an interesting aside, printed electronics has led to the invention of 'plastic memory', which is expected to be extremely low in cost, although the performance might not be as high as the traditional technologies'. Rewritable, non-volatile memories with data retention of around 10 years or one million cycles can be printed using polythiophenes, a family of polymers that display ferroelectric properties. A German company called PolyC GmbH & Co KG has used the technology to make a 20-bit memory using polyethylene terephthalate (PET) as the substrate. Another Norwegian company called Thin Film Electronics ASA is also working in this line. It might be some time before we can buy plastic memory off the shelf, but experts predict there will be a lot of progress in this space, soon.



PARALLEL PROGRAMMING

Four men working in parallel can definitely get more work done than one, but too many cooks can spoil the broth if not managed properly. This is the new twist to the multi-core tale that is emerging now. Multi-core processors are not new anymore — they have emerged victorious even in the desktop and embedded segments, with the industry offering dual-cores, quad-cores and more. The question now is whether developers have the know-how, techniques and most importantly, the right tools required to program the multiple cores and tap their full parallel processing capacity.

Primarily, there needs to be a revolution in the developers' thought process, so that they start thinking and designing software in 'parallel' mode. While some call programming for multi-core as an emerging technology, others argue that it is merely common sense! It does not make sense to develop an application in the traditional, sequential way, and then thrust it on a multi-core processor and expect it to complete the work fast. In fact, doing this might wreak havoc amongst the shared resources.

Developers need to split their programs into parallel tasks intelligently, to ensure proper communication and synchronisation between the various sub-tasks and to avoid software bugs such as race conditions. Plus, developers also need to understand the hardware architecture to a minimal extent at least, to understand how the resources—especially memory—are shared between many cores, and write the application such that the resources are shared optimally. At present, the general sentiment



is that shared memory becomes a bottleneck when there are more than four cores. Such bottlenecks can be avoided if developers sequence and coordinate the sub-tasks in a neat way.

All this is not easy to do, and undeniably, programming for multi-cores and other parallel computing systems is a difficult task. But the challenge is eased to an extent if the developers choose the right concurrent programming languages, libraries, application programming interfaces and parallel programming models.

It is interesting to note that the makers of multi-core and multi-processor platforms, such as Intel, are also making efforts to educate developers on how to write parallel programs and multi-threaded applications. The Parallel Programming resources on Intel's website (<http://software.intel.com/en-us/parallel/>) include not just learning materials but also discussion boards and forums where developers can get their doubts cleared. Initiatives such as OpenCL (an open standard for parallel programming of heterogeneous systems) and Cuda (a parallel computing architecture provided by NVIDIA) are also worth consideration.